JGi JAIN DEEMED-TO-BE UNIVERSITY | ONLINE

Research Project Synopsis
Semester IV

Name	Raghav Gupta
USN	241VMTR01929
Elective	Cyber Security
Date of Submission	21 May 2026

- **Title**

Alert Fatigue Quantifier: A Data-Driven Cognitive Load Monitoring System for SOC Analysts

- **Problem Statement**

Security Operations Centers run on a simple premise: human analysts monitor alerts, investigate threats and respond before damage occurs. In practice, this system fails quietly. Modern SIEM platforms generate thousands of alerts per day, far more than any analyst team can process. The Ponemon Institute found that SOC teams receive an average of 9,854 false positive alerts every week (Ponemon Institute, 2022). SANS Institute surveys show that 66% of SOC teams cannot keep pace with their alert queues (SANS Institute, 2023).

Overwhelmed analysts often cut corners. They close alerts without investigating them and accept severity ratings without verification. Prior research shows that SOC analysts dismiss up to 44% of alerts without investigation during peak fatigue periods (SANS Institute, 2023).

The technical gap is specific: no current cybersecurity tool measures mental fatigue for individual analysts during a live shift. Current tools either reduce alert volume or study burnout later using surveys. Neither approach gives managers real-time warnings when an analyst's decision quality drops. This project builds a continuous early-warning system, tested on simulated operational data that matches fatigue patterns from SANS and Ponemon research.

- **Objectives of the Research**

1. Real-time Fatigue Scoring Framework- To design and evaluate a quantitative model that combines live behavioral signals (alert volume, time-to-triage and escalation deviations) into an Analyst Fatigue Index (AFI). This score is normalized against a 30-day baseline to measure cognitive load objectively.

2. Decision Quality Anomaly Detection- To build and test a method that compares current alert closure patterns against historical baselines. This detects small deviations, like rising false-positive rates, which indicate drops in investigation quality.

3. Predictive Fatigue Modeling- To build a Random Forest classifier that uses alert timing and volume patterns to predict peak fatigue periods. This helps managers rebalance workloads before operational safety limits are reached.

4. Queue Optimization Engine- To build an algorithm that maps fatigue levels to queue adjustments (like temporary alert suppression or optimized shift handovers) to reduce cognitive load without leaving the network vulnerable.

5. Research Contribution- To share a validated, reproducible method for measuring analyst fatigue based on standard logs. This provides the cybersecurity research community with a standardized metric (AFI) for future empirical studies.

- **Research Methodology**

Research Type: Empirical

Data Collection Method: Secondary-Synthetic dataset generated to match statistical distributions from published SOC industry research (SANS Institute, Ponemon Institute, USENIX, IEEE, ACM Computing Surveys)

SDLC Model and Research Approach

This project uses the Agile SDLC model with two-week sprints. Development starts with data analysis and proceeds through system design, algorithm coding and dashboard creation. Agile fits this project because the scoring logic needs early testing against data to refine the formulas based on real

results. A Waterfall model would delay this feedback. The study is empirical: it measures analyst fatigue using simulated log data that matches statistical patterns from published SOC research.

Technology Stack

The tool is built using Python 3.10. Pandas and NumPy clean the data and extract features, while SciPy handles statistical tests like z-score normalization and Mann–Whitney U tests to validate signals. Scikit-Learn runs the Random Forest model to predict fatigue thresholds. Matplotlib and Seaborn generate trend charts. The dashboard is built with Streamlit, which lets users view live scores in a web browser without setting up a web server. Git manages version control and the pipeline processes CSV and JSON files to stay compatible with standard SIEM exports.

Requirement Gathering, System Architecture and Data Flow

System requirements were gathered by reviewing 18 academic and industry publications, including SANS surveys, Ponemon reports and USENIX papers on analyst workload. This review helped identify which behavioral signals show up consistently in fatigue research and confirmed that no real-time, log-based fatigue tracking system currently exists.

The architecture is modular. An ingestion module reads CSV log exports, then a signal module calculates five indicators over 60-minute windows: triage intervals, uninvestigated closures, escalation deviations, enrichment depth and hourly closure rates. The scoring module normalizes these numbers against each analyst's baseline to output a 0–100 fatigue score. A degradation detector compares current false-positive closures to historical baselines, while a prediction module runs the Random Forest model to flag rising fatigue trends. A recommendation module suggests alert suppression and queue adjustments based on the fatigue level. Finally, a Streamlit dashboard displays these outputs. Data flows in a single direction from the raw logs to the dashboard.

Testing, Performance Optimization and Debugging

PyTest runs unit tests for each signal function using custom sample inputs. The Random Forest model is validated with a 70/30 train-test split and k-fold cross-validation to prevent overfitting. We use Python's cProfile module to profile Pandas operations to ensure they process quickly. The VS Code debugger helps trace signal values at each stage and Streamlit's hot-reload feature speeds up dashboard development.

- **Limitation**

Scope: The system operates in a read-only, advisory role. It does not modify live SIEM platforms, ServiceDesk tools, or alert queues. It cannot automatically reassign alerts or change priorities in live workflows.

Data Simulation: The log data is simulated to match published fatigue statistics. Because of this, validation focuses on behavioral patterns rather than actual security incidents. Organizations wanting to use the tool would need to re-validate the scoring system with their own baseline data.

Integration: Live API connections to platforms like Splunk, Sentinel, or QRadar are outside the scope of this 8-week project. The system works with CSV and JSON log exports instead. The recommendation engine only suggests adjustments; it does not change alert rules or queue settings automatically. In addition, because these suggestions are advisory, operational decisions remain entirely with the SOC manager and the tool provides no automated enforcement.

Model Weights: The weights in the scoring formula are based on values from existing literature. Future research should train these weights using supervised learning on larger datasets from real SOC environments.

- **Work Plan (Week 1 to Week 8)**

The following table provides an outline of planned research tasks to be conducted during the eight-week research project period.

Week No.	Activities Completed
Week 1	a) Literature review Phase 1-study 9 sources from SANS, Ponemon and USENIX on SOC alert fatigue patterns and measurement methods. b) Topic finalization: confirm the research gap by verifying that no real-time log-derived AFI system exists in current literature. c) Schema design for the synthetic dataset-define fields like analyst ID, alert ID, triage times, closure types and notes.
Week 2	a) Literature review Phase 2-study 9 additional papers from IEEE, ACM and security conferences on operator workload. b) Draft the synopsis, run a plagiarism check and upload it to the university ERP portal. c) Requirement analysis-define the final AFI signals and the structure of the scoring formula based on the literature.
Week 3	a) System architecture design-create data flow diagrams for the pipeline from log ingestion to the dashboard. b) Derive AFI weights based on significance values found in published studies. c) Document signal definitions-specify the five behavioral indicators and how the rolling 60-minute windows are calculated.
Week 4	a) UML design-draw class and sequence diagrams for all five software modules. b) Design baseline logic for the degradation detector-set threshold calculations for false-positive dismissals. c) Data generation-write scripts to generate synthetic CSV logs that mimic typical SOC shift patterns.
Week 5	a) Build data ingestion module-code the parser, data validation and schema normalization steps. b) Build signal engine-write Pandas window functions and SciPy z-score normalization routines. c) Build scoring module-write the logic for 0–100 scoring and per-analyst baseline calibration. d) Set up Git-initialize the repository and create the folder structure.
Week 6	a) Implement degradation detector-code baseline comparison and anomaly flagging logic. b) Build predictive model-train the Scikit-Learn Random Forest model and set up validation. c) Build recommendation engine-map AFI levels to specific queue adjustments. d) Build Streamlit dashboard-create live gauges, charts and recommendations in the UI.
Week 7	a) Write unit tests-use PyTest to test all signal calculation functions with mock inputs. b) Validate and profile-perform cross-validation on the model and profile code

	performance using cProfile. c) Debug and refine-fix bugs, handle edge cases and polish the Streamlit dashboard. d) Test pipeline-run end-to-end integration tests to verify data flow.
Week 8	a) Draft the final report-write all five chapters according to the university guidelines. b) Compile references-format the 18 sources in APA style and run a plagiarism check. c) Prepare final deliverables-record the five-minute presentation and prepare for the viva.